

Color no animation

```
#include <iostream>
#include <vector>
#include <queue>
#include <cmath>
#include <climits>
#include <algorithm>
```

```
using namespace std;
```

```
// ----- Colors -----
```

```
#define RESET  "\033[0m"
#define GREEN  "\033[32m"
#define RED    "\033[31m"
#define BLUE   "\033[34m"
#define YELLOW "\033[33m"
```

```
// ----- Node Structure -----
```

```
struct Node {
```

```
    int x, y;
```

```
    int g, h, f;
```

```
    Node(int x, int y, int g, int h) {
```

```
        this->x = x;
```

```
        this->y = y;
```

```
        this->g = g;
```

```
        this->h = h;
```

```

        this->f = g + h;
    }

// Min Heap
bool operator<(const Node &other) const {

    return f > other.f;
}

};

// ----- Heuristic Function -----
int heuristic(int x1, int y1,
              int x2, int y2) {

    return abs(x1 - x2) +
           abs(y1 - y2);
}

// ----- Valid Cell Check -----
bool is_valid(int x, int y,
              int n, int m,
              vector<vector<int>> &maze) {

    return (x >= 0 &&
            x < n &&
            y >= 0 &&
            y < m &&
            maze[x][y] == 0);
}

// ----- Print Original Maze -----

```

```

void print_original_maze(
    vector<vector<int>> &maze) {

    int n = maze.size();
    int m = maze[0].size();

    cout << "\nOriginal Maze:\n\n";

    cout << " ";

    for (int j = 0; j < m; j++) {

        cout << j << " ";
    }

    cout << endl;

    for (int i = 0; i < n; i++) {

        cout << i << " | ";

        for (int j = 0; j < m; j++) {

            if (maze[i][j] == 1)
                cout << RED
                    << "1 "
                    << RESET;

            else
                cout << "0 ";
        }
    }
}

```

```

        cout << endl;
    }
}

// ----- Print Final Maze -----
void print_path_matrix(
    vector<vector<int>>> maze,
    vector<pair<int,int>>> &path,
    pair<int,int> start,
    pair<int,int> goal) {

    int n = maze.size();
    int m = maze[0].size();

    // Mark Path
    for (auto &p : path) {

        maze[p.first][p.second] = -1;
    }

    cout << "\nFinal Maze with Path:\n\n";

    cout << "  ";

    for (int j = 0; j < m; j++) {

        cout << j << " ";
    }

    cout << endl;

```

```

for (int i = 0; i < n; i++) {

    cout << i << " | ";

    for (int j = 0; j < m; j++) {

        // Start
        if (i == start.first &&
            j == start.second) {

            cout << BLUE
                << "S "
                << RESET;
        }

        // Goal
        else if (i == goal.first &&
            j == goal.second) {

            cout << YELLOW
                << "G "
                << RESET;
        }

        // Path
        else if (maze[i][j] == -1) {

            cout << GREEN
                << "X "
                << RESET;
        }
    }
}

```

```
}
```

```
// Blocked
```

```
else if (maze[i][j] == 1) {
```

```
    cout << RED
```

```
        << "1 "
```

```
        << RESET;
```

```
}
```

```
// Free Cell
```

```
else {
```

```
    cout << "0 ";
```

```
}
```

```
}
```

```
cout << endl;
```

```
}
```

```
cout << "\nLegend:\n";
```

```
cout << BLUE
```

```
    << "S"
```

```
    << RESET
```

```
    << " = Start\n";
```

```
cout << YELLOW
```

```
    << "G"
```

```
    << RESET
```

```
    << " = Goal\n";
```

```

cout << GREEN

    << "X"

    << RESET

    << " = Path\n";

cout << RED

    << "1"

    << RESET

    << " = Blocked\n";

cout << "0 = Free Cell\n";
}

// ----- Print Final Path -----
void print_path(
    vector<vector<pair<int,int>>> &parent,
    pair<int,int> start,
    pair<int,int> goal,
    vector<vector<int>> &maze) {

    vector<pair<int,int>> path;

    pair<int,int> curr = goal;

    while (curr.first != -1) {

        path.push_back(curr);

        curr = parent[curr.first][curr.second];
    }

```

```
reverse(path.begin(), path.end());
```

```
cout << "\nFinal Path:\n";
```

```
for (auto &p : path) {
```

```
    cout << "("
```

```
        << p.first
```

```
        << ","
```

```
        << p.second
```

```
        << ") ";
```

```
}
```

```
cout << "\nPath Length: "
```

```
    << path.size() - 1
```

```
    << endl;
```

```
print_path_matrix(
```

```
    maze,
```

```
    path,
```

```
    start,
```

```
    goal
```

```
);
```

```
}
```

```
// ----- A* Algorithm -----
```

```
void astar(
```

```
    vector<vector<int>> &maze,
```

```
    pair<int,int> start,
```

```
    pair<int,int> goal) {
```



```
int n = maze.size();
```

```
int m = maze[0].size();
```

```
priority_queue<Node> open_list;
```

```
open_list.push(
```

```
    Node(
```

```
        start.first,
```

```
        start.second,
```

```
        0,
```

```
        heuristic(
```

```
            start.first,
```

```
            start.second,
```

```
            goal.first,
```

```
            goal.second
```

```
        )
```

```
    )
```

```
);
```

```
vector<vector<bool>>> perm_closed(
```

```
    n,
```

```
    vector<bool>(m, false)
```

```
);
```

```
vector<vector<int>>> g_cost(
```

```
    n,
```

```
    vector<int>(m, INT_MAX)
```

```
);
```

```
vector<vector<pair<int,int>>>> parent(
```

```

n,
vector<pair<int,int>>(m, {-1,-1})
);

g_cost[start.first][start.second] = 0;

while (!open_list.empty()) {

    Node current = open_list.top();
    open_list.pop();

    int x = current.x;
    int y = current.y;

    if (perm_closed[x][y])
        continue;

    perm_closed[x][y] = true;

    cout << "\n"
         << BLUE
         << "Current Node: ("
         << x << ", " << y << ") "
         << "g=" << current.g
         << " h=" << current.h
         << " f=" << current.f
         << RESET
         << endl;

    // Goal Check
    if (make_pair(x,y) == goal) {

```

```

cout << GREEN

    << "\nGoal Reached!\n"

    << RESET;

print_path(
    parent,
    start,
    goal,
    maze
);

return;
}

// 4 Directions
vector<pair<int,int>> directions = {

    {-1,0},
    {1,0},
    {0,-1},
    {0,1}
};

for (auto &dir : directions) {

    int nx = x + dir.first;
    int ny = y + dir.second;

    if (is_valid(nx, ny,
        n, m, maze)) {

```

```
int new_g =  
    g_cost[x][y] + 1;
```

```
int h =  
    heuristic(  
        nx,  
        ny,  
        goal.first,  
        goal.second  
    );
```

```
int f = new_g + h;
```

```
cout << "Checking ("  
    << nx << ", " << ny  
    << ") -> "  
    << "g=" << new_g  
    << " h=" << h  
    << " f=" << f;
```

```
if (new_g < g_cost[nx][ny]) {
```

```
    g_cost[nx][ny] = new_g;
```

```
    parent[nx][ny] =  
        {x, y};
```

```
    open_list.push(  
        Node(  
            nx,
```

```
        ny,  
        new_g,  
        h  
    )  
);
```

```
cout << GREEN  
    << " -> ADDED"  
    << RESET  
    << endl;  
}
```

```
else {
```

```
    cout << RED  
        << " -> SKIPPED"  
        << RESET  
        << endl;  
    }  
}  
}  
}
```

```
cout << RED  
    << "\nNo Path Found!\n"  
    << RESET;  
}
```

```
// ----- MAIN -----
```

```
int main() {
```

```
int n, m;
```

```
cout << "Enter rows and columns: ";
```

```
cin >> n >> m;
```

```
vector<vector<int>> maze(  
    n,  
    vector<int>(m)  
);
```

```
cout << "Enter maze "  
    << "(0 = free, 1 = blocked):\n";
```

```
for (int i = 0; i < n; i++) {
```

```
    for (int j = 0; j < m; j++) {
```

```
        cin >> maze[i][j];  
    }  
}
```

```
int sx, sy, gx, gy;
```

```
cout << "Enter start (x y): ";
```

```
cin >> sx >> sy;
```

```
cout << "Enter goal (x y): ";
```

```
cin >> gx >> gy;
```

```
// Print Original Maze
```

```
print_original_maze(maze);
```

```
// Run A*
```

```
astar(  
    maze,  
    {sx, sy},  
    {gx, gy}  
);
```

```
return 0;
```

```
}
```

No Colour

```
#include <iostream>
#include <vector>
#include <queue>
#include <cmath>
#include <limits>
#include <algorithm>
```

```
using namespace std;
```

```
// ----- Node Structure -----
```

```
struct Node {
```

```
    int x, y;
```

```
    int g, h, f;
```

```
    Node(int x, int y, int g, int h) {
```

```
        this->x = x;
```

```
        this->y = y;
```

```
        this->g = g;
```

```
        this->h = h;
```

```
        this->f = g + h;
```

```
    }
```

```
// For Min Heap
```

```
bool operator<(const Node &other) const {
```

```
    return f > other.f;
```

```
}
```

```
};
```



```

// ----- Heuristic Function -----
int heuristic(int x1, int y1, int x2, int y2) {

    return abs(x1 - x2) + abs(y1 - y2);
}

// ----- Valid Cell Check -----
bool is_valid(int x, int y,
              int n, int m,
              vector<vector<int>> &maze) {

    return (x >= 0 && x < n &&
            y >= 0 && y < m &&
            maze[x][y] == 0);
}

// ----- Print Original Maze -----
void print_original_maze(vector<vector<int>> &maze) {

    int n = maze.size();
    int m = maze[0].size();

    cout << "\nOriginal Maze:\n\n";

    // Column numbers
    cout << "  ";
    for (int j = 0; j < m; j++) {
        cout << j << " ";
    }

```

```

cout << endl;
for (int i = 0; i < n; i++) {

    cout << i << " | ";

    for (int j = 0; j < m; j++) {

        cout << maze[i][j] << " ";

    }

    cout << endl;
}
}

// ----- Print Final Maze with Path -----
void print_path_matrix(vector<vector<int>> maze,
                      vector<pair<int,int>> &path,
                      pair<int,int> start,
                      pair<int,int> goal) {

    int n = maze.size();
    int m = maze[0].size();

    // Mark Path
    for (auto &p : path) {

        int x = p.first;
        int y = p.second;

        maze[x][y] = -1;
    }
}

```

```
cout << "\nFinal Maze with Path:\n\n";
```

```
// Column numbers
```

```
cout << "  ";
```

```
for (int j = 0; j < m; j++) {
```

```
    cout << j << " ";
```

```
}
```

```
cout << endl;
```

```
for (int i = 0; i < n; i++) {
```

```
    cout << i << " | ";
```

```
for (int j = 0; j < m; j++) {
```

```
    // Start Node
```

```
    if (i == start.first &&
```

```
        j == start.second) {
```

```
        cout << "S ";
```

```
    }
```

```
    // Goal Node
```

```
    else if (i == goal.first &&
```

```
        j == goal.second) {
```

```
        cout << "G ";
```

```
    }
```

```

// Path
else if (maze[i][j] == -1) {

    cout << "X ";
}

// Blocked Cell
else if (maze[i][j] == 1) {

    cout << "1 ";
}

// Free Cell
else {

    cout << "0 ";
}
}

cout << endl;
}

cout << "\nLegend:\n";
cout << "S = Start\n";
cout << "G = Goal\n";
cout << "X = Path\n";
cout << "1 = Blocked\n";
cout << "0 = Free Cell\n";
}

// ----- Print Final Path -----

```

```
void print_path(vector<vector<pair<int,int>>> &parent,  
               pair<int,int> start,  
               pair<int,int> goal,  
               vector<vector<int>> &maze) {
```

```
    vector<pair<int,int>> path;
```

```
    pair<int,int> curr = goal;
```

```
    while (curr.first != -1) {
```

```
        path.push_back(curr);
```

```
        curr = parent[curr.first][curr.second];
```

```
    }
```

```
    reverse(path.begin(), path.end());
```

```
    cout << "\nFinal Path:\n";
```

```
    for (auto &p : path) {
```

```
        cout << "("
```

```
            << p.first
```

```
            << ","
```

```
            << p.second
```

```
            << ") ";
```

```
    }
```

```
    cout << "\nPath Length: "
```

```
        << path.size() - 1
```

```

        << endl;

// Print highlighted maze
print_path_matrix(
    maze,
    path,
    start,
    goal
);
}

// ----- A* Algorithm -----
void astar(vector<vector<int>> &maze,
    pair<int,int> start,
    pair<int,int> goal) {

    int n = maze.size();
    int m = maze[0].size();

    priority_queue<Node> open_list;

    open_list.push(
        Node(
            start.first,
            start.second,
            0,
            heuristic(
                start.first,
                start.second,
                goal.first,
                goal.second
            )
        )
    );
}

```

```
    )  
    )  
};
```

```
vector<vector<bool>> perm_closed(  
    n,  
    vector<bool>(m, false)  
);
```

```
vector<vector<int>> g_cost(  
    n,  
    vector<int>(m, INT_MAX)  
);
```

```
vector<vector<pair<int,int>>> parent(  
    n,  
    vector<pair<int,int>>(m, {-1,-1})  
);
```

```
g_cost[start.first][start.second] = 0;
```

```
while (!open_list.empty()) {
```

```
    Node current = open_list.top();  
    open_list.pop();
```

```
    int x = current.x;  
    int y = current.y;
```

```
    if (perm_closed[x][y])  
        continue;
```

```
perm_closed[x][y] = true;
```

```
cout << "\nCurrent Node: ("
```

```
    << x << ", " << y << ") "
```

```
    << "g=" << current.g
```

```
    << " h=" << current.h
```

```
    << " f=" << current.f
```

```
    << endl;
```

```
// Goal Check
```

```
if (make_pair(x,y) == goal) {
```

```
    cout << "\nGoal Reached!\n";
```

```
    print_path(
```

```
        parent,
```

```
        start,
```

```
        goal,
```

```
        maze
```

```
    );
```

```
    return;
```

```
}
```

```
// 4 Directions
```

```
vector<pair<int,int>> directions = {
```

```
    {-1,0},
```

```
    {1,0},
```

```
    {0,-1},
```



```

        {0,1}
    };

    for (auto &dir : directions) {

        int nx = x + dir.first;
        int ny = y + dir.second;

        if (is_valid(nx, ny, n, m, maze)) {

            int new_g = g_cost[x][y] + 1;

            int h = heuristic(
                nx,
                ny,
                goal.first,
                goal.second
            );

            int f = new_g + h;

            cout << "Checking ("
                << nx << "," << ny
                << ") -> "
                << "g=" << new_g
                << " h=" << h
                << " f=" << f;

            if (new_g < g_cost[nx][ny]) {

                g_cost[nx][ny] = new_g;
            }
        }
    }
}

```

```

        parent[nx][ny] = {x, y};

        open_list.push(
            Node(nx, ny, new_g, h)
        );

        cout << " -> ADDED\n";
    }

    else {

        cout << " -> SKIPPED\n";
    }
}

}

}

}

}

cout << "\nNo Path Found!\n";
}

// ----- MAIN -----
int main() {

    int n, m;

    cout << "Enter rows and columns: ";
    cin >> n >> m;

    vector<vector<int>> maze(
        n,

```

```

        vector<int>(m)
    );

    cout << "Enter maze (0 = free, 1 = blocked):\n";

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < m; j++) {

            cin >> maze[i][j];
        }
    }

    int sx, sy, gx, gy;

    cout << "Enter start (x y): ";
    cin >> sx >> sy;

    cout << "Enter goal (x y): ";
    cin >> gx >> gy;

    // Print original maze
    print_original_maze(maze);

    // Run A*
    astar(
        maze,
        {sx, sy},
        {gx, gy}
    );

```

```
return 0;
```

```
}
```